

II. Heuristic Methods

We continue with our problem of optimization.

Throughout, Ω is a closed and bounded subset of $\mathbb{R}^n$, and $f: \Omega \rightarrow \mathbb{R}$ a continuous function.

We call Ω the *feasible domain*, and f the *objective function*, and write the problem as follows:

$$\begin{array}{ll} \text{Maximize / minimize} & f(x) \\ \text{Subject to} & x \in \Omega \end{array}$$

The Extreme Value Theorem guarantees:

f takes a maximum and a minimum value on Ω .

In optimization, one often finds difficulties such as:

- the objective function f is not differentiable,
- there is no exact formula for the objective function. Instead there is an algorithm that computes the value of the function at each point.
- One works in a high dimensional space, and the common algorithms require too much memory and computation time (complexity).

Heuristic algorithms can be used in such situations. They are algorithms designed to find *approximate solutions* by “trial and error”, that is, by *random methods*. The algorithms use strategies to improve performance which often are inspired by natural processes, or by animal behavior, for example.

II.1 Naïve Random Search

Naïve random search algorithms are totally random, without strategy.

a) Basic naïve random search

Algorithm:

1. Pick a collection of sample points $x_1, \dots, x_N$ in Ω by random.
2. Evaluate the objective function at these points.
3. The smallest/largest of the values of $f(x_1), \dots, f(x_N)$ will be the max/min.

Advantages (of the basic naïve random search):

- A very simple algorithm
- Does not require much memory
- Computations can be done in parallel

Disadvantage:

For good results, one needs an extremely large number N of sample points, which makes this algorithm extremely slow. This is particularly true in high dimensions.

```

# example of naïve random search for minimization in one dimension
from numpy import arange
from numpy.random import rand
from matplotlib import pyplot

# Parameters:
N_samples = 100
# objective function
def f(x):
    return x**2.0

# the feasible domain Omega is an interval [r_min,r_max]
r_min, r_max = -5.0, 5.0
# generate a random sample of N_samples points from the domain
sample = r_min + rand( N_samples) * (r_max - r_min)
# evaluate the sample
sample_eval = f(sample)
#
# locate the best solution within sample_eval
best_ix = 0
for i in range(len(sample)):
    if sample_eval[i] < sample_eval[best_ix]:
        best_ix = i
#
# summarize best solution
print('Best: f(%.5f) = %.5f' % (sample[best_ix], sample_eval[best_ix]))

```

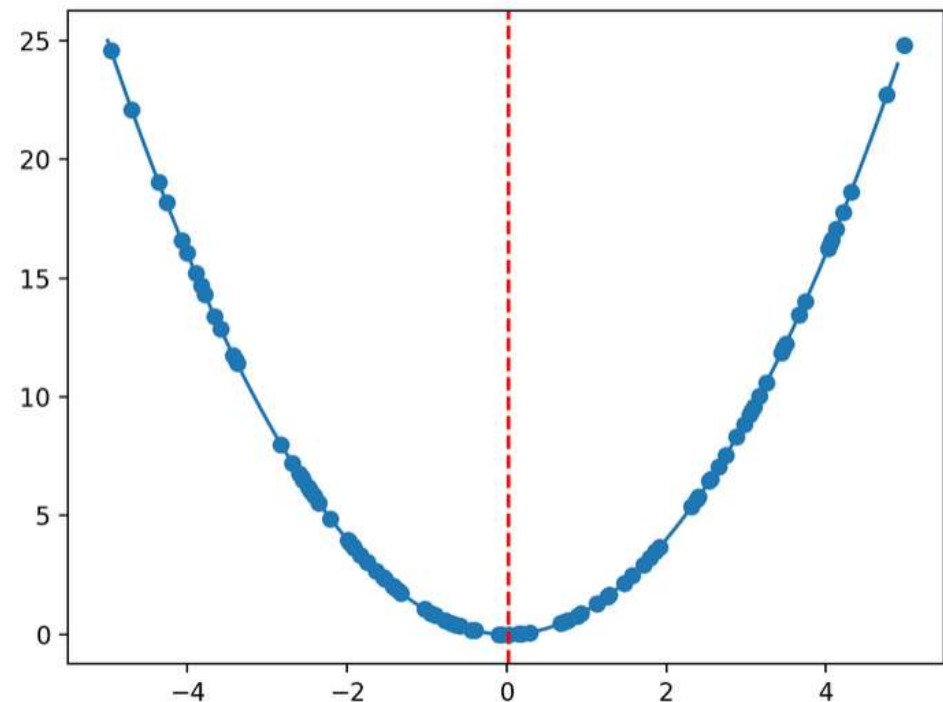
```
# get ready to plot
# sample input range uniformly at 0.1 increments
inputs = arange(r_min, r_max, 0.1)
# compute targets
results = f(inputs)
# create a line plot of input vs result
pyplot.plot(inputs, results)
# plot the sample
pyplot.scatter(sample, sample_eval)
# draw a vertical line at the best input
pyplot.axvline(x=sample[best_ix], ls='--', color='red')
pyplot.show()
```

Objective function:

$$f(x, y) = x^2$$

Minimum found:

$$f(-0.00704) = 0.00005$$



b) Grid Search (There is no randomness in this algorithm)

Instead of choosing the sample points $x_1, \dots, x_N$ by random, they are distributed over the the feasible domain Ω in a regular way:

Algorithm:

1. Place a uniform grid $x_1, \dots, x_N$ over Ω .
2. Evaluate the objective function at these grid points.
3. The smallest/largest of the values of $f(x_1), \dots, f(x_N)$ will be the max/min.

Advantages:

- This is a very simple algorithm. Easy to implement.
- Does not require much memory.
- Computations can be done in parallel.
- One can use grid search iteratively. In a first step, identify regions in Ω which are 'good'. Then place a finer grid over the good regions and repeat the search.

Disadvantages:

- If the grid is too coarse, one may miss optimal solutions.
- The number of grid points grows exponentially with the dimension n . Thus, in high dimensions, a fine grid results in a large number of grid points, which can make the algorithm slow.

```

# example of grid search for function minimization in a 2d-problem
from numpy import arange
from numpy import meshgrid
from numpy.random import rand
from matplotlib import pyplot

# objective function
def f(x, y):
    return x**2.0 + y**2.0

# define the feasible region Omega (a square)
r_min, r_max = -5.0, 5.2
# generate a grid sample from the domain
sample = list()
step = 0.4
for x in arange(r_min, r_max+step, step):
    for y in arange(r_min, r_max+step, step):
        sample.append([x,y])
# evaluate the sample
sample_eval = [f(x,y) for x,y in sample]
# locate the best solution
best_ix = 0
for i in range(len(sample)):
    if sample_eval[i] < sample_eval[best_ix]:
        best_ix = i
# summarize best solution
print( 'Best: f(%.5f,%.5f) = %.5f' % (sample[best_ix][0],
                                     sample[best_ix][1], sample_eval[best_ix]) )

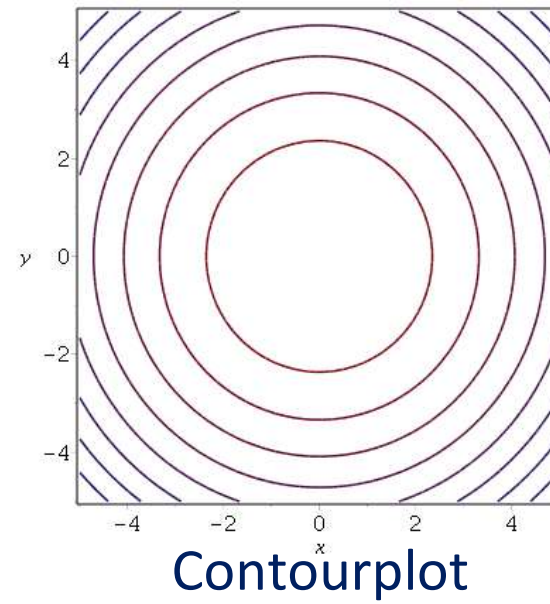
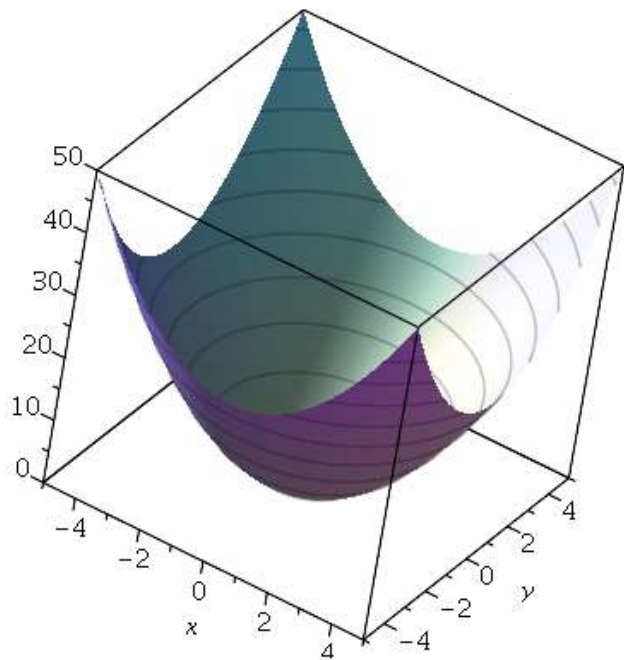
```

```
##### ready to plot
#
# create a mesh uniformly at 0.1 increments
xaxis = arange(r_min, r_max, 0.1)
yaxis = arange(r_min, r_max, 0.1)
x, y = meshgrid(xaxis, yaxis)
# compute targets
results = f(x, y)
# create a filled contour plot
my_contourplot = pyplot.contourf(x, y, results, levels=50, cmap='jet')
pyplot.colorbar(my_contourplot, label='Z values')
# plot the gridpoints as black circles
pyplot.plot([x for x,_ in sample], [y for _,y in sample], '.', color='black')
# draw the best result as a white star
pyplot.plot(sample[best_ix][0], sample[best_ix][1], '*', color='white')
# show the plot
pyplot.show()
```

Results: Objective function: $f(x, y) = x^2 + y^2$

We have imposed a mesh on the search space $\Omega = [-5, 5.2] \times [-5, 5.2]$ with $\Delta x = \Delta y = 0.4$. Therefore, the exact “minimizer” $(x, y) = (0, 0)$ is NOT a mesh point.

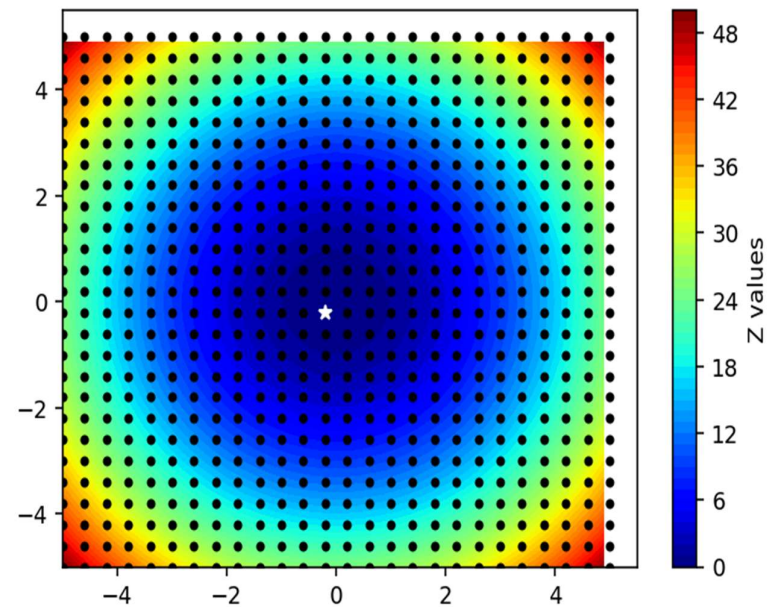
Minimum found: $f(-0.20000, -0.20000) = 0.08000$



3d-plot with contourlines

$$f(x) = x^2 + y^2$$

contourplot
with colormap



c) **Basic Neighborhood (local) naïve random search algorithm:**

In this algorithm, the points $x_1, \dots, x_N$ are chosen *sequentially*:

Given any sample point x_k , the next sample point x_{k+1} is selected randomly from within a neighborhood $N(x_k)$ of x_k .

Algorithm:

1. Set $k = 0$ and pick an initial x_0 in Ω .
2. Pick a candidate z_k close to x_k , at random.
(z_k is from a neighborhood $N(x_k)$ of x_k .)
3. If $f(z_k) < f(x_k)$, then set $x_{k+1} = z_k$, else set $x_{k+1} = x_k$.
4. If the stopping criterion is satisfied, stop. $f(x_{k+1})$ is the optimal value.
5. Set $k := k + 1$ and go to step 2.

Advantages: (when compared to naïve random search)

- We need search across the whole search space, but come closer to a point of minimum with each iteration
- If the objective function f is “nice”, fewer sample points x_k are required.

Disadvantage:

- We may end up at a **local** minimum, not an **absolute** minimum
- Computations can NOT be done in parallel.

```

# example of neighborhood naive random search
import numpy as np
from numpy import arange
from numpy.random import rand
from matplotlib import pyplot

# objective function
def f(x):
    return x**2.0

# the feasible domain Omega is an interval [r_min,r_max]
r_min, r_max = -5.0, 5.0
# size of neighborhood: [x_k-eps,x_k+eps]
eps = 1.0;
# stopping criterion: number of iterations
n_iter = 100
# record all the sample points for plotting
sample_points=np.zeros( n_iter )
# initial sample point x_0
x_k = r_min + rand() * (r_max - r_min)    # uniform distribution
# Loop to construct sequence of sample points
for k in range( n_iter ):
    # case of uniform distribution
    z_k = x_k - eps + rand() * 2*eps
    # case of standard normal distribution
    # z_k = x_k + np.random.normal()
    # test whether z_k is outside the feasible domain:
    if (z_k < r_min):

```

```

        z_k = r_min
    if (z_k > r_max ):
        z_k = r_max;
    if f(z_k) < f(x_k):
        x_k = z_k
# record all the sample points for plotting
sample_points[ k ] = x_k
#
# summarize best solution
print('Best: f(%.5f) = %.5f' % ( x_k, f(x_k) ) )
#
# get ready to plot
# sample input range uniformly at 0.1 increments
inputs = arange(r_min, r_max, 0.1)
# compute targets
results = f(inputs)
# create a line plot of input vs result
pyplot.plot(inputs, results)
# plot the sample points/values that we have moved through
sample_eval = f( sample_points )
pyplot.scatter(sample_points, sample_eval)
# draw a vertical line at the best input
pyplot.axvline(x=x_k, ls='--', color='red')
pyplot.show()
# print the sequence of sample points to show convergence towards the
location of the min
print(np.array_str( sample_points , precision=5, suppress_small=True))

```

Example: $f(x) = x^2$

Starting point: $x_0=3.83249$ (by random)

Choose z_k in

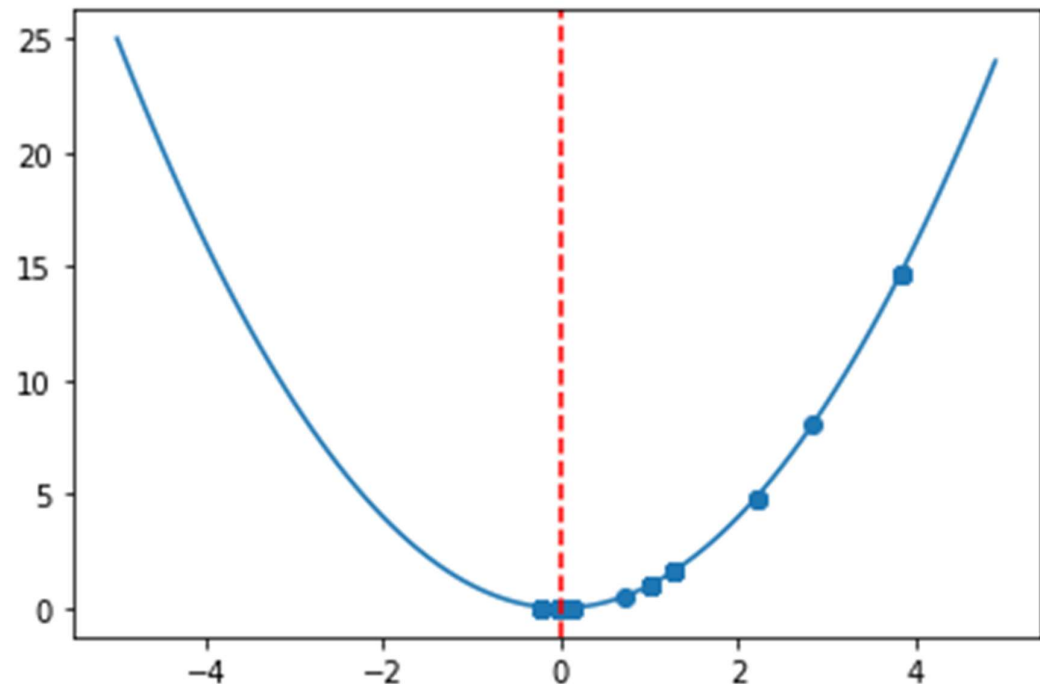
$$[x_k - \epsilon, x_k + \epsilon]$$

uniformly distributed

Minimum found:

$$f(-0.01062) = 0.00011$$

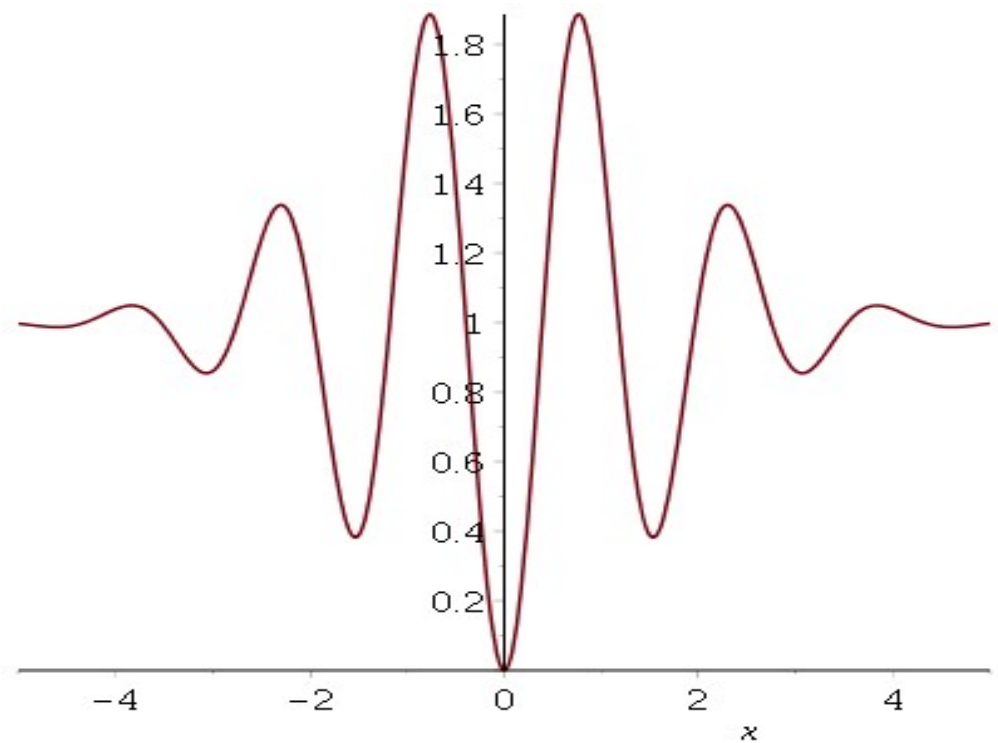
70 iterations



```
[ 3.83249  3.83249  3.83249  3.83249  2.84005  2.2031  2.2031  2.2031
 1.27975  1.27975  1.27975  1.27975  1.27975  1.27975  1.27975  1.02065
 1.02065  1.02065  1.02065  1.02065  1.02065  1.02065  1.02065  0.72901
-0.21257 -0.21257 -0.21257 -0.21257 -0.21257 -0.21257 -0.21257  0.13497
 0.13497  0.13497  0.13497  0.13497 -0.01914 -0.01914 -0.01914 -0.01914
-0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914
-0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01914
-0.01914 -0.01914 -0.01914 -0.01914 -0.01914 -0.01062 -0.01062 -0.01062
-0.01062 -0.01062 -0.01062 -0.01062 -0.01062 -0.01062 ]
```

Example:

$$f(x) = 1 - e^{-0.2x^2} \cos(4x)$$



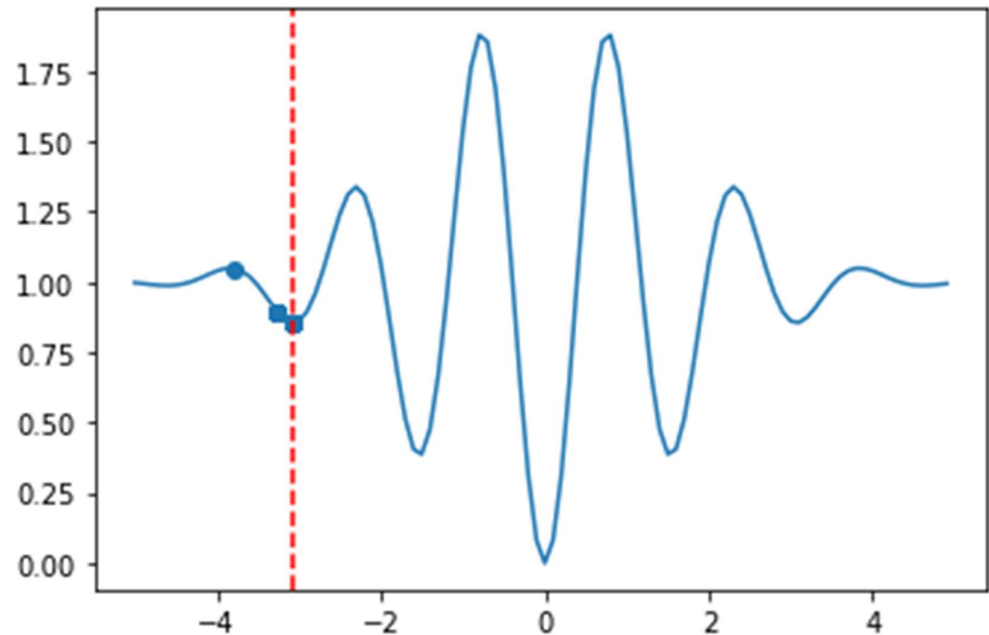
This function has many local max/min

The following example chooses z_k in the neighborhood $[x_k - 1, x_k + 1]$.

Starting point: $x_0 = -3.78477$

Minimum found:

$f(-3.05873) = 0.85443$



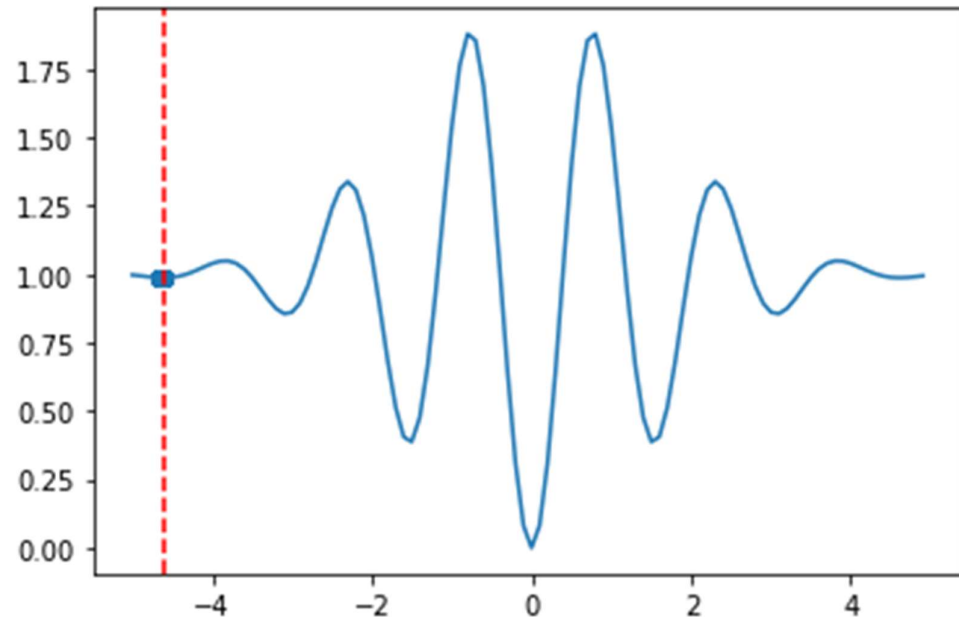
80 iterations (n_iter=80):

```
[ -3.78477  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686
  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686  -3.25686
  -3.25686  -3.25686  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873
  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873
  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873
  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873
  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873
  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873  -3.05873 ]
```

Starting point: $x_0 = -4.65463$

Minimum found:

$f(-4.60498) = 0.98692$



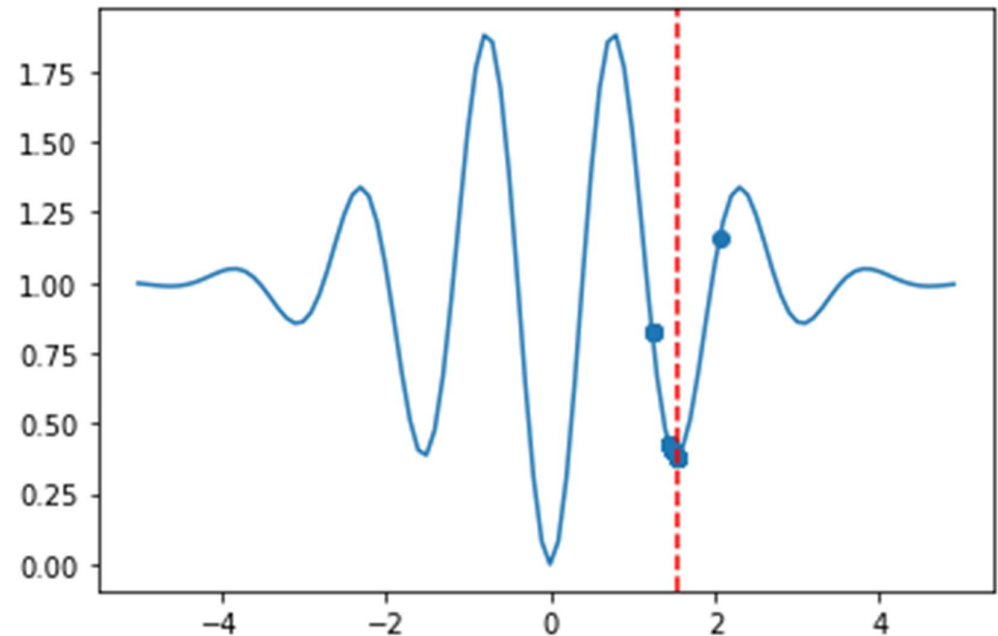
80 iterations (n_iter=80):

```
[ -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463
  -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463
  -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.62414 -4.61860
  -4.61860 -4.61860 -4.61860 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
  -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
  -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
  -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
  -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
  -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 ]
```

Starting point: $x_0=2.05952$

Minimum found

$f(1.53602) = 0.38219$



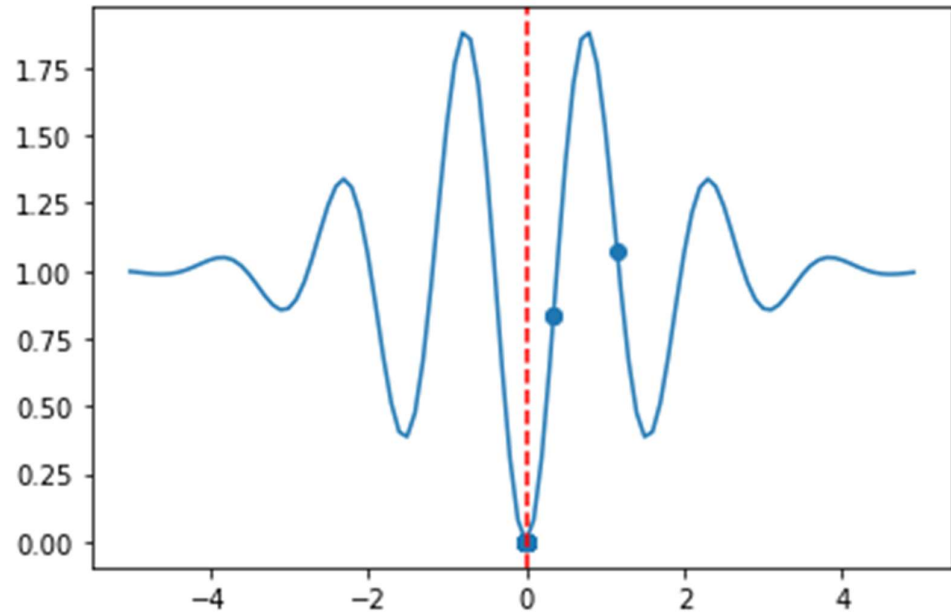
80 iterations (n_iter=80):

```
[2.05952 1.23798 1.23798 1.23798 1.23798 1.44574 1.44574 1.44574 1.44574
 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574
 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.46421 1.46421
 1.46421 1.46421 1.46421 1.46421 1.46421 1.46421 1.46421 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 ]
```

Starting point: $x=0.47310$

Minimum found

$f(0.00152) = 0.00002$



80 iterations (n_iter=80):

```
[ 1.15379  0.35128  0.35128 -0.0094  -0.0094  -0.0094  -0.0094  -0.0094
 -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094
 -0.0094  -0.0094  -0.0094  -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152
 0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152 ]
```

Question: How can we avoid ending up at a **local** minimum/maximum in the “basic neighborhood naïve random search algorithm” ?

Some possible solutions are:

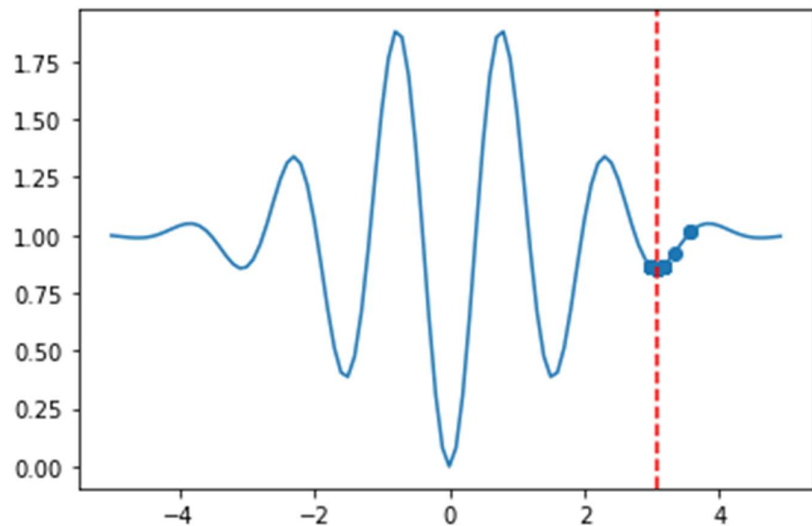
1. Allow the next sample point z_k to be far from x_k (i.e. make the neighbourhood $N(x_k)$ large).
(This may require a larger number of iterations to be performed, because one may jump across the whole feasible domain= search space)
2. Allow the next sample point z_k to be far from x_k , but with only low probability.
3. **Repeated Search.** Repeat the search algorithm several times, each time with a different starting point.
4. Allow that x_{k+1} be a “worse” point than x_k , with some restrictions.
(in a minimization problem, “worse” means $f(x_{k+1}) > f(x_k)$)
(e.g. the *Simulated Annealing* algorithm. (which we will study)

5. Introduce many “search agents” into the basic neighborhood naïve random search algorithm

Let's look at these possibilities by examples.

1. Make the neighbourhood $N(x_k)$ large.

i) Search neighborhood: $z_k \in [x_k - 1, x_k + 1]$, uniform distribution



Starting point: $x_0 = 4.0$

$x_1 = 3.57106$

$x_3 = 3.32028$

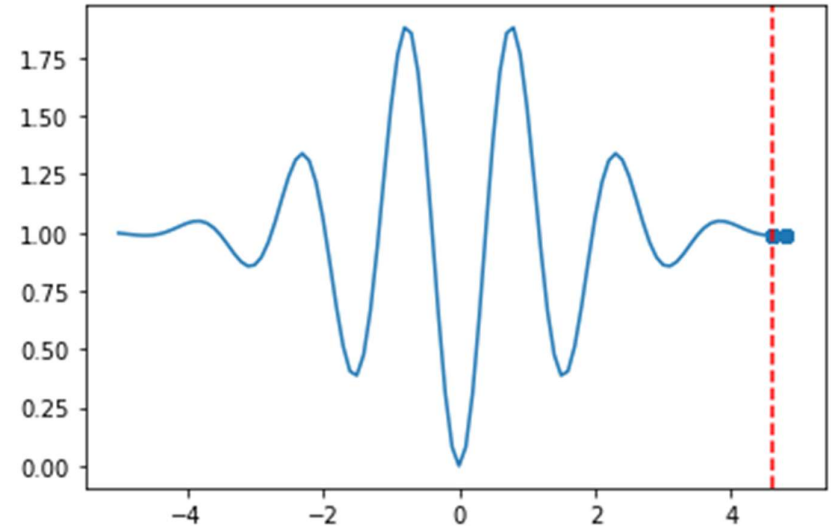
$x_4 = 2.96592$

$x_{19} = 3.17664$

$x_{33} = 3.16511$

$x_{57} = 3.07443$

Minimum found: $f(3.07443) = 0.85441$



Starting point: $x_0 = 4.00000$

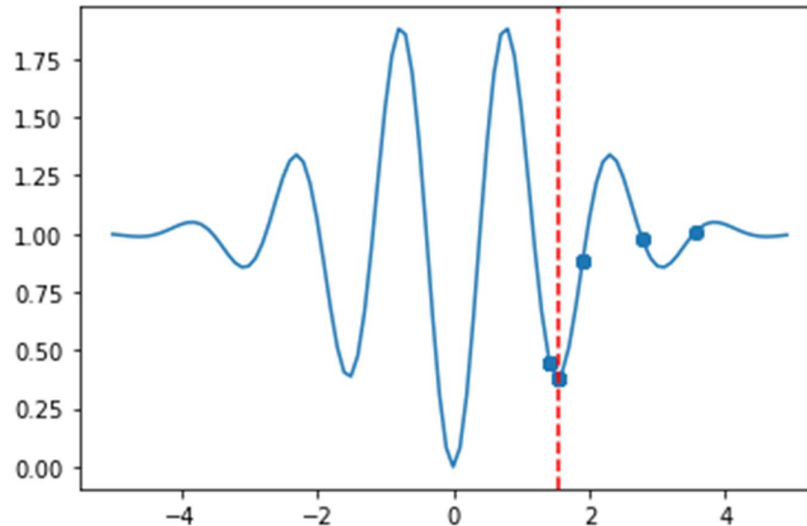
$x_1 = 4.80698$

$x_6 = 4.59634$

Minimum found: $f(4.59634) = 0.98693$

Typical possible solution

The worst possible solution



Starting point: $x_0 = 4.0$

$x_1 = 3.54576$

$x_3 = 2.77038$

$x_6 = 1.9046$

$x_{11} = 1.42397$

$x_{21} = 1.54584$

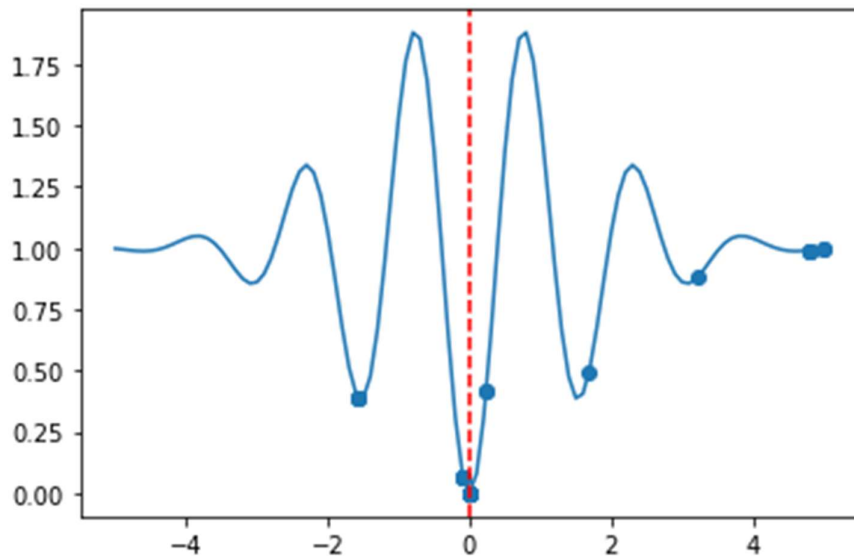
$x_{50} = 1.53659$

Minimum found: $f(1.53659) = 0.38221$

The best possible solution is only a local min

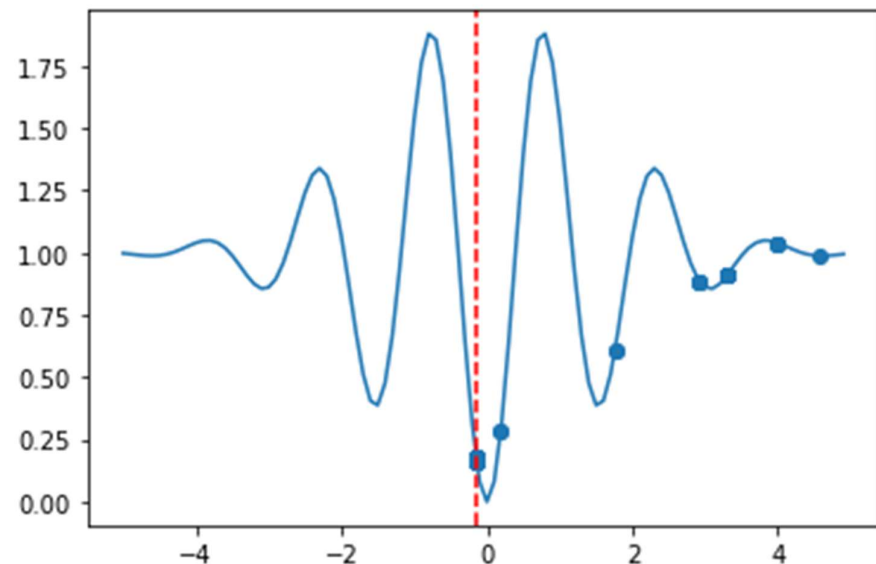
(one can never reach the absolute min. point at $x=0$
because the neighborhood $z_k \in [x_k - 1, x_k + 1]$ is too small)

ii) Enlarge the search neighborhood to $z_k \in [x_k - 2, x_k + 2]$,
uniform distribution



Starting point: $x_0=4.0$

$x_1 = 5.0$
 $x_3 = 4.79653$
 $x_7 = 3.23482$
 $x_8 = 1.68551$
 $x_9 = 0.23388$
 $x_{11} = -1.5604$



Starting point: $x_0=4.0$

$x_1 = 4.0$
 $x_5 = 4.55422$
 $x_6 = 3.29548$
 $x_{14} = 2.91816$
 $x_{17} = 1.7586$
 $x_{19} = 0.19188$

x_15= -0.09326
x_20= -0.01138
x_40= -0.00738

Minimum found: $f(-0.00738) = 0.00045$

Move over a large region in search space

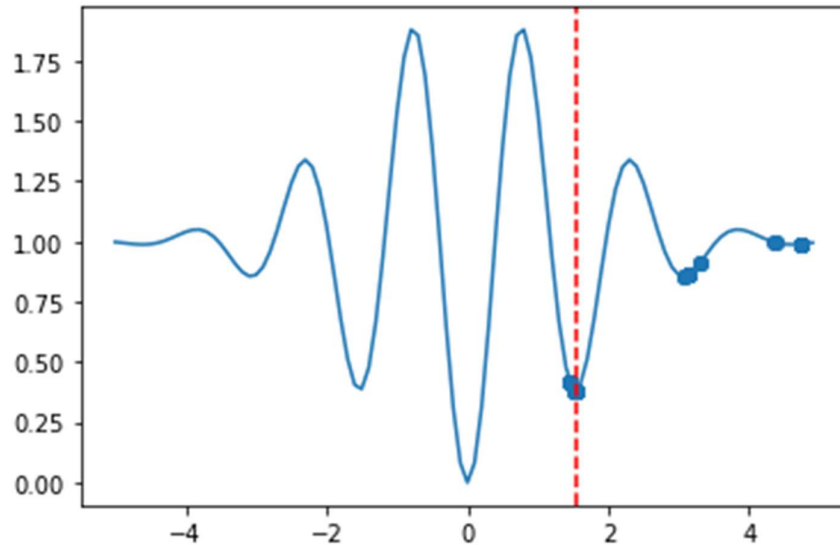
x_21= -0.14938
x_75= -0.14302

Minimum found: $f(-0.14302) = 0.16266$

Solution of low precision

In 50 runs, the absolute min was always reached.
However, sometimes with low precision.

2. Allow z_k to be far from x_k , but with only low probability.
 (Choose z_k with Gaussian distribution, mean= x_k , SD=0.8)

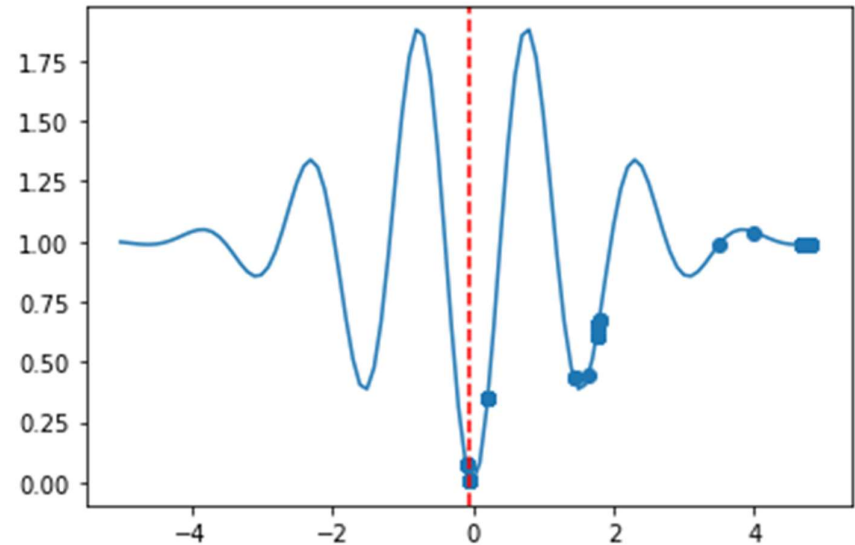


Starting point: $x_0=4.0$

$x_1 = 4.32696$
 $x_2 = 4.37061$
 $x_4 = 4.71978$
 $x_{14} = 3.30022$
 $x_{17} = 3.13442$
 $x_{42} = 3.07127$
 $x_{45} = 1.44985$
 $x_{53} = 1.5156$
 $x_{58} = 1.54762$

Minimum found: $f(1.54762) = 0.38327$

Local min only



Starting point: $x_0=4.0$

$x_1 = 4.0$
 $x_2 = 4.78117$
 $x_7 = 4.7127$
 $x_{16} = 4.69391$
 $x_{28} = 4.67959$
 $x_{38} = 3.4943$
 $x_{39} = 1.79788$
 $x_{42} = 1.78633$
 $x_{55} = 1.75859$
 $x_{61} = 1.64667$
 $x_{62} = 1.4327$

```
x_65= 0.21555  
x_78= -0.09363  
x_85= -0.0369  
Minimum found: f(-0.03690) = 0.01114
```

With a choice of SD=0.8, the absolute min is reached in **most of 100 runs**

With a choice of SD=1.0, the absolute min is reached in **all runs**

Possible modification: decrease the SD with each iteration

3. Repeated Search.

Assignment: Modify the code of

example of neighborhood naive random search

by looping over starting points -4.5, -3.5, -2.5, 2.5, 3.5, 4.5

Then select the best of the solutions.

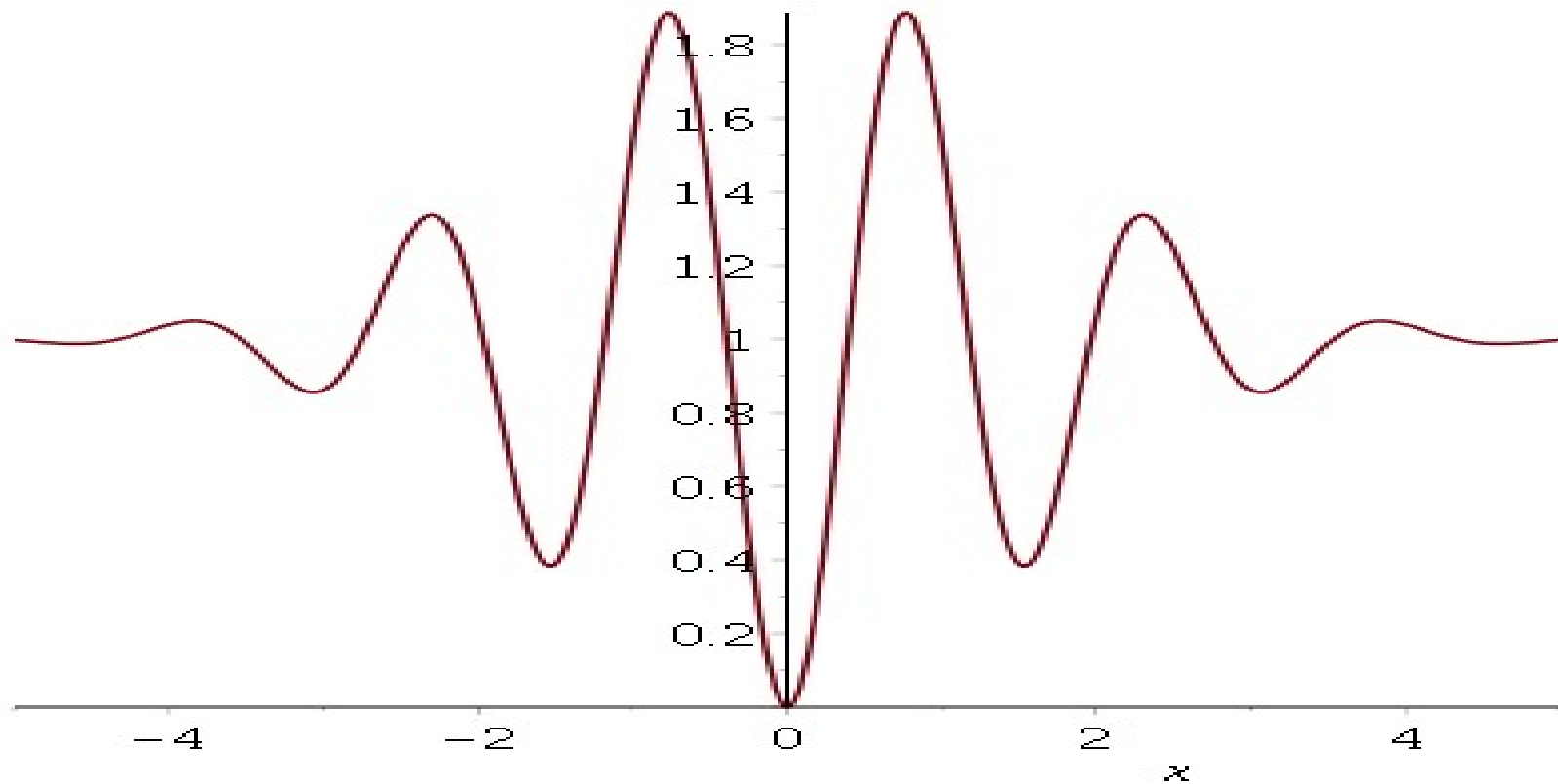
4. Simulated Annealing

Modify the basic neighborhood naive random search:

As before, when $f(z_k) \leq f(x_k)$ then we set $x_{k+1} = z_k$

However, if $f(z_k) > f(x_k)$ then we toss a coin whose probability of (HEAD) equals $p = p(k, x_k, z_k)$.

If the toss results in HEAD, then we set $x_{k+1} = z_k$, (so that $f(x_{k+1}) > f(x_k)$) otherwise we keep $x_{k+1} = x_k$.



Note: i) Because $f(x_{k+1}) > f(x_k)$ is allowed, we need to keep track of the *best* x_k so far achieved in the algorithm; call it x_{best} .

- ii) The probability $p = p(k, x_k, z_k)$ of obtaining a HEAD will depend on
- the iteration k ,
 - the value of $f(x_k)$ and
 - the value $f(z_k)$.

A common choice is

$$p = p(k, x_k, z_k) = \min(1, e^{-[f(z_k) - f(x_k)]/T_k})$$

where

$$T_k = \frac{T_0}{k+1} \quad (\text{"temperature"})$$

and T_0 is a problem-dependent constant.

We see: During the first iterations (k small), T_k is relatively large, so that $-\frac{[f(z_k) - f(x_k)]}{T_k}$ is close to 0; thus $p = e^{-[f(z_k) - f(x_k)]/T_k}$ is close to 1, and the probability of a HEAD is large.

As k increases, the probability p of a HEAD decreases towards zero.

Also, when $f(z_k)$ is much larger than $f(x_k)$, then

$-[f(z_k) - f(x_k)]/T_k$ is large negative,

so that the probability $p = e^{-[f(z_k) - f(x_k)]/T_k}$ of a HEAD is small.

Other choices for T_k are:

$$T_k = \frac{T_0}{\ln(k+2)},$$

$$T_k = \alpha^k T_0, \quad 0 < \alpha < 1 \text{ fixed}$$

Simulated Annealing Algorithm (for minimization)

1. Set $k = 0$ and pick an initial x_0 in Ω , $x_{\text{best}} = x_0$.
2. Pick a candidate z_k close to x_k , at random.
(z_k is from a neighborhood $N(x_k)$ of x_k .)
3. If $f(z_k) < f(x_k)$ then set $x_{k+1} = z_k$, $x_{\text{best}} = z_k$,
else { compute the probability $p = p(k, x_k, z_k)$
(coin toss:) pick a number r , $0 \leq r \leq 1$, (unif. distr.)
If $r < p$ (“HEAD”) then $x_{k+1} = z_k$,
else (“TAIL”) $x_{k+1} = x_k$ }
4. If the stopping criterion is satisfied, stop. $f(x_{\text{best}})$ is the optimal value.
5. Set $k := k + 1$ and go to step 2.

```
x_0 = 4.0
```

```
>1 f([4.00854361]) = 1.03809
```

```
>4 f([4.31634496]) = 1.00032
```

```
>5 f([4.46008078]) = 0.99003
```

```
>6 f([4.46080896]) = 0.99000
```

```
>7 f([4.64264068]) = 0.98710
```

```
>8 f([4.6174176]) = 0.98694
```

```
>128 f([3.23913579]) = 0.88657
```

```
>129 f([3.09792609]) = 0.85554
```

```
>138 f([1.88303743]) = 0.84436
```

```
>141 f([1.72355168]) = 0.54784
```

```
>142 f([1.566317]) = 0.38788
```

```
>156 f([0.17438372]) = 0.23822
```

```
>158 f([-0.0859482]) = 0.05991
```

```
>172 f([0.01610672]) = 0.00213
```

```
>237 f([0.01472293]) = 0.00178
```

```
>255 f([0.00664156]) = 0.00036
```

```
>277 f([-0.00033045]) = 0.00000
```

```
>991 f([3.77760799e-05]) = 0.00000
```

Minimum found: $f([3.77760799e-05]) = 0.000000$

